| | | |
|---|---|---|
| Language Translation and Execution | 2 | 3 |
| Program Abstraction and Representation | | 3 |
| Syntax Analysis | | |
| Compiler Semantic Analysis | | |
| Program Analysis and Analyzers | | |
| Code Generation | | |
| Runtime Behavior and Systems | | |
| Advanced Programming Constructs | | |
| Language Pragmatics | | |
| Formal Semantics | | |
| Formal Development Methodologies | | |
| Design Principles of Programming Languages | | |
| Society, Ethics, and the Profession | Included in SEP hours | |
| **Total** | **21** | **19** |

The CS and KA Core totals do not include the shared hours that have been counted in other knowledge areas.

## Knowledge Units

### FPL-OOP: Object-Oriented Programming

***CS Core:***

1. Imperative programming as a subset of object-oriented programming.
2. Object-oriented design:
   a. Decomposition into objects carrying state and having behavior.
   b. Class-hierarchy design for modeling.
3. Definition of classes: fields, methods, and constructors. (See also: SDF-Fundamentals)
4. Subclasses, inheritance (including multiple inheritance), and method overriding.
5. Dynamic dispatch: definition of method-call.
6. Exception handling. (See also: SDF-Fundamentals, PDC-Coordination, SE-Construction)
7. Object-oriented idioms for encapsulation:
   a. Privacy, data hiding, and visibility of class members.
   b. Interfaces revealing only method signatures.
   c. Abstract base classes, traits and mixins.

8. Dynamic vs static properties.
9. Composition vs inheritance.
10. Subtyping:
    a. Subtype polymorphism; implicit upcasts in typed languages.
    b. Notion of behavioral replacement: subtypes acting like supertype.
    c. Relationship between subtyping and inheritance.

***KA Core:***

11. Collection classes, iterators, and other common library components.
12. Metaprogramming and reflection.

***Illustrative Learning Outcomes:***

***CS Core:***

1. Enumerate the differences between imperative and object-oriented programming paradigms.
2. Compose a class through design, implementation, and testing to meet behavioral requirements.
3. Build a simple class hierarchy utilizing subclassing that allows code to be reused for distinct subclasses.
4. Predict and validate control flow in a program using dynamic dispatch.
5. Compare and contrast how computational solutions to a problem differ in procedural, functional, and object-oriented approaches.
6. Compare and contrast mechanisms to define and protect data elements within procedural, functional, and object-oriented approaches.
7. Compare and contrast the benefits and costs/impact of using inheritance (subclasses) and composition (specifically, how to base composition on higher order functions).
8. Explain the relationship between object-oriented inheritance (code-sharing and overriding) and subtyping (the idea of a subtype being usable in a context that expects the supertype).
9. Use object-oriented encapsulation mechanisms such as interfaces and private members.
10. Define and use iterators and other operations on aggregates, including operations that take functions as arguments, in multiple programming languages, selecting the most natural idioms for each language. (See also: FPL-Functional)

***KA Core:***

11. Use collection classes and iterators effectively to solve a problem.

### FPL-Functional: Functional Programming

***CS Core:***

1. Lambda expressions and evaluation: (See also: AL-Models, FPL-Formalism)
    a. Variable binding and scope rules. (See also: SDF-Fundamentals)
    b. Parameter-passing. (See also: SDF-Fundamentals)
    c. Nested lambda expressions and reduction order.
2. Effect-free programming:
    a. Function calls have no side effects, facilitating compositional reasoning.
    b. Immutable variables and data copying vs reduction.
    c. Use of recursion vs loops vs pipelining (map/reduce).

3. Processing structured data (e.g., trees) via functions with cases for each data variant:
   a. Functions defined over compound data in terms of functions applied to the constituent pieces.
   b. Persistent data structures.
4. Using higher-order functions (taking, returning, and storing functions).

***KA Core:***

5. Metaprogramming and reflection.
6. Function closures (functions using variables in the enclosing lexical environment).
   a. Basic meaning and definition – creating closures at run-time by capturing the environment.
   b. Canonical idioms: call-backs, arguments to iterators, reusable code via function arguments.
   c. Using a closure to encapsulate data in its environment.
   d. Delayed versus eager evaluation.

***Non-core:***

7. Graph reduction machine and call-by-need.
8. Implementing delayed evaluation.
9. Integration with logic programming paradigm using concepts such as equational logic, narrowing, residuation and semantic unification. (See also: FPL-Logic)
10. Integration with other programming paradigms such as imperative and object-oriented.

***Illustrative learning outcomes:***

***CS Core:***

1. Develop basic algorithms that avoid assigning to mutable states or considering reference equality.
2. Develop useful functions that take and return other functions.
3. Compare and contrast how computational solutions to a problem differ in procedural, functional, and object-oriented approaches.
4. Compare and contrast mechanisms to define and protect data elements within procedural, functional, and object-oriented approaches.

***KA Core:***

5. Explain a simple example of lambda expression being implemented using a virtual machine, such as a SECD machine, showing storage and reclaim of the environment.
6. Correctly interpret variables and lexical scope in a program using function closures.
7. Use functional encapsulation mechanisms such as closures and modular interfaces.
8. Compare and contrast stateful vs stateless execution.
9. Define and use iterators and other operations on aggregates, including operations that take functions as arguments, in multiple programming languages, selecting the most natural idioms for each language. (See also: FPL-OOP)

***Non-core:***

10. Illustrate graph reduction using a λ-expression using a shared subexpression.
11. Illustrate the execution of a simple nested λ-expression using an abstract machine, such as an ABC machine.
12. Illustrate narrowing, residuation, and semantic unification using simple illustrative examples.

13. Illustrate the concurrency constructs using simple programming examples of known concepts such as a buffer being read and written concurrently or sequentially. (See also: FPL-OOP)

### FPL-Logic: Logic Programming

***KA Core:***

1. Universal vs existential quantifiers. (See also: AI-LRR, MSF-Discrete)
2. First order predicate logic vs higher order logic. (See also: AI-LRR, MSF-Discrete)
3. Expressing complex relations using logical connectives and simpler relations.
4. Definitions of Horn clause, facts, goals and subgoals.
5. Unification and unification algorithm; unification vs assertion vs expression evaluation.
6. Mixing relations with functions. (See also: MSF-Discrete)
7. Cuts, backtracking, and non-determinism.
8. Closed-world vs open-world assumptions.

***Non-core:***

9. Memory overhead of variable copying in handling iterative programs.
10. Programming constructs to store partial computation and pruning search trees.
11. Mixing functional programming and logic programming using concepts such as equational logic, narrowing, residuation, and semantic unification. (See also: FPL-Functional)
12. Higher-order, constraint, and inductive logic programming. (See also: AI-LRR)
13. Integration with other programming paradigms such as object-oriented programming.
14. Advance programming constructs such as difference-lists, creating user defined data structures, set of, etc.

***Illustrative learning outcomes:***

***KA Core:***

1. Use a logic language to implement a conventional algorithm.
2. Use a logic language to implement an algorithm employing implicit search using clauses, relations, and cuts.
3. Use a simple illustrative example to show correspondence between First Order Predicate Logic (FOPL) and logic programs using Horn clauses.
4. Use examples to illustrate the unification algorithm and its role of parameter-passing in query reduction.
5. Use simple logic programs interleaving relations, functions, and recursive programming such as factorial and Fibonacci numbers and simple complex relationships between entities and illustrate execution and parameter-passing using unification and backtracking.

***Non-core:***

6. Illustrate computation of simple programs such as Fibonacci and show overhead of recomputation, and then show how to improve execution overhead.

### FPL-Scripting: Shell Scripting

***CS Core:***

1. Error/exception handling
2. Piping (See also: AR-Organization, SF-Overview, OS-Process)
3. System commands (See also: SF-Overview)
    a. Interface with operating systems (See also: SF-Overview, OS-Principles)
4. Environment variables (See also: SF-Overview)
5. File abstraction and operators (See also: SDF-Fundamentals, OS-Files, SF-Resource)
6. Data structures, such as arrays and lists. (See also: AL-Foundational, SDF-Fundamentals, SDF-Data-Structures)
7. Regular expressions (See also: AL-Models)
8. Programs and processes (See also: OS-Process)
9. Workflow

***Illustrative learning outcomes:***

***CS Core:***

1. Create and execute automated scripts to manage various system tasks.
2. Solve various text processing problems through scripting.

#### FPL-Event-Driven: Event-Driven and Reactive Programming

***CS Core:***

1. Procedural programming vs reactive programming: advantages of reactive programming in capturing events.
2. Components of reactive programming: event-source, event signals, listeners and dispatchers, event objects, adapters, event-handlers. (See also: GIT-Interaction, SPD-Web, SPD-Mobile, SPD-Robot, SPD-Embedded, SPD-Game, SPD-Interactive)
3. Stateless and state-transition models of event-based programming.
4. Canonical uses such as GUIs, mobile devices, robots, servers. (See also: GIT-Interaction, GIT-Image, SPD-Web, SPD-Mobile, SPD-Robot, SPD-Embedded, SPD-Game, SPD-Interactive)

***KA Core:***

5. Using a reactive framework:
    a. Defining event handlers/listeners
    b. Parameterization of event senders and event arguments
    c. Externally generated events and program-generated events
6. Separation of model, view, and controller
7. Event-driven and reactive programs as state-transition systems

***Illustrative learning outcomes:***

***CS Core:***

1. Implement event handlers for use in reactive systems, such as GUIs.
2. Examine why an event-driven programming style is natural in domains where programs react to external events.

***KA Core:***

3. Define and use a reactive framework.

4. Describe an interactive system in terms of a model, a view, and a controller.

### FPL-Parallel: Parallel and Distributed Computing

***CS Core:***

1. Safety and liveness (See also: PDC-Evaluation)
   a. Race conditions (See also: OS-Concurrency)
   b. Dependencies/preconditions
   c. Fault models (See also: OS-Faults)
   d. Termination (See also: PDC-Coordination)
2. Programming models (See also: PDC-Programs)
   One or more of the following:
   a. Actor models
   b. Procedural and reactive models
   c. Synchronous/asynchronous programming models
   d. Data parallelism
3. Properties (See also: PDC-Programs, PDC-Coordination)
   a. Order-based properties
      i. Commutativity
      ii. Independence
   b. Consistency-based properties
      i. Atomicity
      ii. Consensus
4. Execution control: (See also: PDC-Coordination, SF-Foundations)
   a. Async await
   b. Promises
   c. Threads
5. Communication and coordination (See also: OS-Process, PDC-Communication, PDC-Coordination)
   a. Mutexes
   b. Message-passing
   c. Shared memory
   d. Cobegin-coend
   e. Monitors
   f. Channels
   g. Threads
   h. Guards

***KA Core:***

6. Futures
7. Language support for data parallelism such as forall, loop unrolling, map/reduce
8. Effect of memory-consistency models on language semantics and correct code generation
9. Representational State Transfer Application Programming Interfaces (REST APIs)
10. Technologies and approaches: cloud computing, high performance computing, quantum computing, ubiquitous computing
11. Overheads of message-passing

12. Granularity of program for efficient exploitation of concurrency
13. Concurrency and other programming paradigms (e.g., functional)

***Illustrative learning outcomes:***

***CS Core:***

1. Explain why programming languages do not guarantee sequential consistency in the presence of data races and what programmers must do as a result.
2. Implement correct concurrent programs using multiple programming models, such as shared memory, actors, futures, synchronization constructs, and data-parallelism primitives.
3. Use a message-passing model to analyze a communication protocol.
4. Use synchronization constructions such as monitor/synchronized methods in a simple program.
5. Modeling data dependency using simple programming constructs involving variables, read and write.
6. Modeling control dependency using simple constructs such as selection and iteration.

***KA Core:***

7. Explain how REST API's integrate applications and automate processes.
8. Explain benefits, constraints and challenges related to distributed and parallel computing.

#### FPL-Aspect: Aspect-Oriented Programming

***Non-core:***

1. Aspects
2. Join points
3. Advice
   a. Before
   b. After (as finally, returning or throwing)
   c. Around
4. Point cuts
   a. Designators
5. Weaving – static and dynamic
6. Alternatives including annotations and IDEs

#### FPL-Types: Type Systems

***CS Core:***

1. A type as a set of values together with a set of operations
   a. Primitive types (e.g., numbers, Booleans) (See also: SDF-Fundamentals)
   b. Compound types built from other types (e.g., records/structs, unions, arrays, lists, functions, references using set operations) (See also: SDF-Data-Structures)
2. Association of types to variables, arguments, results, and fields
3. Type safety as an aspect of program correctness (See also: FPL-Formalism)
4. Type safety and errors caused by using values inconsistently given their intended types
5. Goals and limitations of static and dynamic typing: detecting and eliminating errors as early as possible.

6. Generic types (parametric polymorphism)
    a. Definition and advantages of polymorphism: parametric, subtyping, overloading, and coercion
    b. Comparison of monomorphic and polymorphic types
    c. Comparison with ad-hoc polymorphism (overloading) and subtype polymorphism
    d. Generic parameters and typing
    e. Use of generic libraries such as collections
    f. Comparison with ad hoc polymorphism (overloading) and subtype polymorphism
    g. Prescriptive vs descriptive polymorphism
    h. Implementation models of polymorphic types
    i. Subtyping

***KA Core:***

7. Type equivalence: structural vs name equivalence
8. Complementary benefits of static and dynamic typing:
    a. Errors early vs errors late/avoided
    b. Enforce invariants during code development and code maintenance vs postpone typing decisions while prototyping and conveniently allow flexible coding patterns such as heterogeneous collections.
    c. Typing rules for function, product, and sum types
    d. Avoiding misuse of code vs allowing more code reuse
    e. Detect incomplete programs vs allow incomplete programs to run
    f. Relationship to static analysis
    g. Decidability

***Non-core:***

9. Compositional type constructors, such as product types (for aggregates), sum types (for unions), function types, quantified types, and recursive types
10. Type checking
11. Subtyping: (See also: FPL-OOP)
    a. Subtype polymorphism; implicit upcasts in typed languages
    b. Notion of behavioral replacement: subtypes acting like supertype
    c. Relationship between subtyping and inheritance
12. Type safety as preservation plus progress
13. Type inference
14. Static overloading
15. Propositions as types (implication as a function, conjunction as a product, disjunction as a sum) (See also: FPL-Formalism)
16. Dependent types (universal quantification as dependent function, existential quantification as dependent product). (See also: FPL-Formalism)

***Illustrative learning outcomes:***

***CS Core:***

1. Describe, for both a primitive and a compound type, the values that have that type.

2. Describe, for a language with a static type system, the operations that are forbidden statically, such as passing the wrong type of value to a function or method.
3. Describe examples of program errors detected by a type system.
4. Identify program properties, for multiple programming languages, that are checked statically and program properties that are checked dynamically.
5. Describe an example program that does not type-check in a particular language and yet would have no error if run.
6. Use types and type-error messages to write and debug programs.

***KA Core:***

7. Explain how typing rules define the set of operations that are legal for a type.
8. List the type rules governing the use of a particular compound type.
9. Explain why undecidability requires type systems to conservatively approximate program behavior.
10. Define and use program pieces (such as functions, classes, methods) that use generic types, including for collections.
11. Discuss the differences among generics, subtyping, and overloading.
12. Explain multiple benefits and limitations of static typing in writing, maintaining, and debugging software.

***Non-core:***

13. Define a type system precisely and compositionally.
14. For various foundational type constructors, identify the values they describe and the invariants they enforce.
15. Precisely describe the invariants preserved by a sound type system.
16. Prove type safety for a simple language in terms of preservation and progress theorems.
17. Implement a unification-based type-inference algorithm for a simple language.
18. Explain how static overloading and associated resolution algorithms influence the dynamic behavior of programs.

### FPL-Systems: Systems Execution and Memory Model

***CS Core:***

1. Data structures for translation, execution, translation, and code mobility such as stack, heap, aliasing (sharing using pointers), indexed sequence and string
2. Direct, indirect, and indexed access to memory location
3. Run-time representation of data abstractions such as variables, arrays, vectors, records, pointer-based data elements such as linked-lists and trees, and objects
4. Abstract low-level machine with simple instruction, stack, and heap to explain translation and execution
5. Run-time layout of memory: activation record (with various pointers), static data, call-stack, heap (See also: AR-Memory, OS-Memory)
   a. Translating selection and iterative constructs to control-flow diagrams
   b. Translating control-flow diagrams to low level abstract code
   c. Implementing loops, recursion, and tail calls

   d. Translating function/procedure calls and return from calls, including different parameter-passing mechanisms using an abstract machine
6. Memory management: (See also: AR-Memory, OS-Memory)
   a. Low level allocation and accessing of high-level data structures such as basic data types, n-dimensional array, vector, record, and objects
   b. Return from procedure as automatic deallocation mechanism for local data elements in the stack
   c. Manual memory management: allocating, de-allocating, and reusing heap memory
   d. Automated memory management: garbage collection as an automated technique using the notion of reachability
7. Green computing. (See also: SEP-Sustainability)

***Illustrative learning outcomes:***

***CS Core:***

1. Explain how a core language construct, such as data abstractions and control abstractions, is executed.
2. Explain how programming language implementations typically organize memory into global data, text, heap, and stack sections and how features such as recursion and memory management map to this memory model.
3. Explain why memory leaks and dangling pointer problems occur, and what can be done by a programmer to avoid/fix them.

#### FPL-Translation: Language Translation and Execution

***CS Core:***

1. Execution models for JIT (Just-In-Time), compiler, interpreter
2. Use of intermediate code, e.g., bytecode
3. Limitations and benefits of JIT, compiler, and interpreter
4. Cross compilers/transpilers
5. BNF and extended BNF representation of context-free grammar
6. Parse tree using a simple sentence such as arithmetic expression or if-then-else statement
7. Execution as native code or within a virtual machine
8. Language translation pipeline: syntax analysis, parsing, optional type-checking, translation/code generation and optimization, linking, loading, execution

***KA Core:***

9. Run-time representation of core language constructs such as objects (method tables) and functions that can be passed as parameters to and returned from functions (closures)
10. Secure compiler development (See also: SEC-Foundations, SEC-Coding)

***Illustrative learning outcomes:***

***CS Core:***

1. Explain and understand the differences between compiled, JIT, and interpreted language implementations, including the benefits and limitations of each.
2. Differentiate syntax and parsing from semantics and evaluation.

3. Use BNF and extended BNF to specify the syntax of simple constructs such as if-then-else, type declaration and iterative constructs for known languages such as C++ or Python.
4. Illustrate the parse tree using a simple sentence/arithmetic expression.
5. Illustrate translation of syntax diagrams to BNF/extended BNF for simple constructs such as if-then-else, type declaration, iterative constructs, etc.
6. Illustrate ambiguity in parsing using nested if-then-else/arithmetic expression and show resolution using precedence order.

***KA-Core:***

7. Discuss the benefits and limitations of garbage collection, including the notion of reachability.

#### FPL-Abstraction: Program Abstraction and Representation

***KA Core:***

1. BNF and regular expressions
2. Programs that take (other) programs as input such as interpreters, compilers, type-checkers, documentation generators
3. Components of a language:
   a. Definitions of alphabets, delimiters, sentences, syntax, and semantics
   b. Syntax vs semantics
4. Program as a set of non-ambiguous meaningful sentences
5. Basic programming abstractions: constants, variables, declarations (including nested declarations), command, expression, assignment, selection, definite and indefinite iteration, iterators, function, procedure, modules, exception handling (See also: SDF-Fundamentals)
6. Mutable vs immutable variables: advantages and disadvantages of reusing existing memory location vs advantages of copying and keeping old values; storing partial computation vs recomputation
7. Types of variables: static, local, nonlocal, global; need and issues with nonlocal and global variables.
8. Scope rules: static vs dynamic; visibility of variables; side-effects.
9. Side-effects induced by nonlocal variables, global variables and aliased variables.

***Non-core:***

10. L-values and R-values: mapping mutable variable-name to L-values; mapping immutable variable-names to R-values
11. Environment vs store and their properties
12. Data and control abstraction
13. Mechanisms for information exchange between program units such as procedures, functions, and modules: nonlocal variables, global variables, parameter-passing, import-export between modules
14. Data structures to represent code for execution, translation, or transmission.
15. Low level instruction representation such as virtual machine instructions, assembly language, and binary representation (See also: AR-Representation, AR-Assembly)
16. Lambda calculus, variable binding, and variable renaming. (See also: AL-Models, FPL-Formalism)
17. Types of semantics: operational, axiomatic, denotational, behavioral; define and use abstract syntax trees; contrast with concrete syntax.

***Illustrative learning outcomes:***

***KA Core:***

1. Illustrate the scope of variables and visibility using simple programs.
2. Illustrate different types of parameter-passing using simple pseudo programming language.
3. Explain side-effect using global and nonlocal variables and how to fix such programs.
4. Explain how programs that process other programs treat the other programs as their input data.
5. Describe a grammar and an abstract syntax tree for a small language.
6. Describe the benefits of having program representations other than strings of source code.
7. Implement a program to process some representation of code for some purpose, such as an interpreter, an expression optimizer, or a documentation generator.

### FPL-Syntax: Syntax Analysis

***Non-core:***

1. Regular grammars vs context-free grammars (See also: AL-Models)
2. Scanning and parsing based on language specifications
3. Lexical analysis using regular expressions
4. Tokens and their use
5. Parsing strategies including top-down (e.g., recursive descent, or LL) and bottom-up (e.g., LR or GLR) techniques
    a. Lookahead tables and their application to parsing
6. Language theory:
    a. Chomsky hierarchy (See also: AL-Models)
    b. Left-most/right-most derivation and ambiguity
    c. Grammar transformation
7. Parser error recovery mechanisms
8. Generating scanners and parsers from declarative specifications

***Illustrative learning outcomes:***

***Non-core:***

1. Use formal grammars to specify the syntax of languages.
2. Illustrate the role of lookahead tables in parsing.
3. Use declarative tools to generate parsers and scanners.
4. Recognize key issues in syntax definitions: ambiguity, associativity, precedence.

### FPL-Semantics: Compiler Semantic Analysis

***Non-core:***

1. Abstract syntax trees; contrast with concrete syntax
2. Defining, traversing, and modifying high-level program representations
3. Scope and binding resolution
4. Static semantics
    a. Type checking.
    b. Define before use

   c. Annotation and extended static checking frameworks.
5. L-values/R-values (See also: SDF-Fundamentals)
6. Call semantics
7. Types of parameter-passing with simple illustrations and comparison: call by value, call by reference, call by value-result, call by name, call by need and their variations
8. Declarative specifications such as attribute grammars and their applications in handling limited context-base grammar

***Illustrative learning outcomes:***

***Non-core:***

1. Draw the abstract syntax tree for a small language.
2. Implement context-sensitive, source-level static analyses such as type-checkers or resolving identifiers to identify their binding occurrences.
3. Describe semantic analyses using an attribute grammar.

### FPL-Analysis: Program Analysis and Analyzers

***Non-core:***

4. Relevant program representations, such as basic blocks, control-flow graphs, def-use chains, and static single assignment
5. Undecidability and consequences for program analysis
6. Flow-insensitive analysis, such as type-checking and scalable pointer and alias analysis
7. Flow-sensitive analysis, such as forward and backward dataflow analyses
8. Path-sensitive analysis, such as software model checking and software verification
9. Tools and frameworks for implementing analyzers
10. Role of static analysis in program optimization and data dependency analysis during exploitation of concurrency (See also: FPL-Code)
11. Role of program analysis in (partial) verification and bug-finding (See also: FPL-Code)
12. Parallelization:
    a. Analysis for auto-parallelization
    b. Analysis for detecting concurrency bugs

***Illustrative learning outcomes:***

***Non-core:***

1. Explain the difference between dataflow graph and control flow graph.
2. Explain why non-trivial sound program analyses must be approximate.
3. Argue why an analysis is correct (sound and terminating).
4. Explain why potential aliasing limits sound program analysis and how alias analysis can help.
5. Use the results of a program analysis for program optimization and/or partial program correctness.

### FPL-Code: Code Generation

***Non-core:***

1. Instruction sets (See also: AR-Assembly)
2. Control flow

3. Memory management (See also: AR-Memory, OS-Memory)
4. Procedure calls and method dispatching
5. Separate compilation; linking
6. Instruction selection
7. Instruction scheduling (e.g., pipelining)
8. Register allocation
9. Code optimization as a form of program analysis (See also: FPL-Analysis)
10. Program generation through generative AI

***Illustrative learning outcomes:***

***Non-core:***

1. Identify all essential steps for automatically converting source code into assembly or other low-level languages.
2. Explain the low-level code necessary for calling functions/methods in modern languages.
3. Discuss why separate compilation requires uniform calling conventions.
4. Discuss why separate compilation limits optimization because of unknown effects of calls.
5. Discuss opportunities for optimization introduced by naive translation and approaches for achieving. optimization, such as instruction selection, instruction scheduling, register allocation, and peephole optimization.

### FPL-Run-Time: Run-time Behavior and Systems

***Non-core:***

1. Process models using stacks and heaps to allocate and deallocate activation records and recovering environments using frame pointers and return addresses during a procedure call including parameter-passing examples
2. Schematics of code lookup using hash tables for methods in implementations of object-oriented programs
3. Data layout for objects and activation records
4. Object allocation in heap
5. Implementing virtual entities and virtual methods; virtual method tables and their application
6. Run-time behavior of object-oriented programs
7. Compare and contrast allocation of memory during information exchange using parameter-passing and non-local variables (using chain of static links).
8. Dynamic memory management approaches and techniques: malloc/free, garbage collection (mark-sweep, copying, reference counting), regions (also known as arenas or zones)
9. Just-in-time compilation and dynamic recompilation
10. Interface to operating system (e.g., for program initialization)
11. Interoperability between programming languages including parameter-passing mechanisms and data representation (See also: AR-Representation)
    a. Big endian, little endian
    b. Data layout of composite data types such as arrays
12. Other common features of virtual machines, such as class loading, threads, and security checking
13. Sandboxing

***Illustrative learning outcomes:***

***Non-core:***

1. Discuss benefits and limitations of automatic memory management.
2. Explain the use of metadata in run-time representations of objects and activation records, such as class pointers, array lengths, return addresses, and frame pointers.
3. Compare and contrast static allocation vs stack-based allocation vs heap-based allocation of data elements.
4. Explain why some data elements cannot be automatically deallocated at the end of a procedure/method call (need for garbage collection).
5. Discuss advantages, disadvantages, and difficulties of just-in-time and dynamic recompilation.
6. Discuss the use of sandboxing in mobile code.
7. Identify the services provided by modern language run-time systems.

### FPL-Constructs: Advanced Programming Constructs

***Non-core:***

1. Encapsulation mechanisms
2. Delayed evaluation and infinite streams
3. Compare and contrast delayed evaluation vs eager evaluation
4. Unification vs assertion vs expression evaluation
5. Control abstractions: exception handling, continuations, monads.
6. Object-oriented abstractions: multiple inheritance, mixins, traits, multimethods
7. Metaprogramming: macros, generative programming, model-based development
8. String manipulation via pattern-matching (regular expressions)
9. Dynamic code evaluation ("eval")
10. Language support for checking assertions, invariants, and pre/post-conditions
11. Domain specific languages, such as database languages, data science languages, embedded computing languages, synchronous languages, hardware interface languages
12. Massive parallel high performance computing models and languages

***Illustrative learning outcomes:***

***Non-core:***

1. Use various advanced programming constructs and idioms correctly.
2. Discuss how various advanced programming constructs aim to improve program structure, software quality, and programmer productivity.
3. Discuss how various advanced programming constructs interact with the definition and implementation of other language features.

### FPL-Pragmatics: Language Pragmatics

***Non-core:***

1. Effect of technology needs and software requirements on programming language development and evolution
2. Problem domains and programming paradigm
3. Criteria for good programming language design

   a. Principles of language design such as orthogonality
   b. Defining control and iteration constructs
   c. Modularization of large software
4. Evaluation order, precedence, and associativity
5. Eager vs delayed evaluation
6. Defining control and iteration constructs
7. External calls and system libraries

***Illustrative learning outcomes:***

***Non-core:***

1. Discuss the role of concepts such as orthogonality and well-chosen defaults in language design.
2. Objectively evaluate and justify language-design decisions.
3. Implement an example program whose result can differ under different rules for evaluation order, precedence, or associativity.
4. Illustrate uses of delayed evaluation, such as user-defined control abstractions.
5. Discuss the need for allowing calls to external calls and system libraries and the consequences for language implementation.

#### FPL-Formalism: Formal Semantics

***Non-core:***

1. Syntax vs semantics
2. Approaches to semantics: axiomatic, operational, denotational, type-based
3. Axiomatic semantics of abstract constructs such as assignment, selection, iteration using pre-condition, post-conditions, and loop invariant
4. Operational semantics analysis of abstract constructs and sequence of such as assignment, expression evaluation, selection, iteration using environment and store
   a. Symbolic execution
   b. Constraint checkers
5. Denotational semantics
   a. Lambda Calculus. (See also: AL-Models, FPL-Functional)
6. Proofs by induction over language semantics
7. Formal definitions and proofs for type systems (See also: FPL-Types)
   a. Propositions as types (implication as a function, conjunction as a product, disjunction as a sum)
   b. Dependent types (universal quantification as dependent function, existential quantification as dependent product)
   c. Parametricity

***Illustrative learning outcomes:***

***Non-core:***

1. Construct formal semantics for a small language.
2. Write a lambda-calculus program and show its evaluation to a normal form.
3. Discuss the different approaches of operational, denotational, and axiomatic semantics.
4. Use induction to prove properties of all programs in a language.

5. Use induction to prove properties of all programs in a language that is well-typed according to a formally defined type system.
6. Use parametricity to establish the behavior of code given only its type.

### FPL-Methodologies: Formal Development Methodologies

1. Formal specification languages and methodologies
2. Theorem provers, proof assistants, and logics
3. Constraint checkers (See also: FPL-Formalism)
4. Dependent types (universal quantification as dependent function, existential quantification as dependent product) (See also: FPL-Types, FPL-Formalism)
5. Specification and proof discharge for fully verified software systems using pre/post conditions, refinement types, etc.
6. Formal modeling and manual refinement/implementation of software systems.
7. Use of symbolic testing and fuzzing in software development.
8. Model checking.
9. Understanding of situations where formal methods can be effectively applied and how to structure development to maximize their value.

***Illustrative learning outcomes:***

***Non-core:***

1. Use formal modeling techniques to develop and validate architectures.
2. Use proof assisted programming languages to develop fully specified and verified software artifacts.
3. Use verifier and specification support in programming languages to formally validate system properties.
4. Integrate symbolic validation tooling into a programming workflow.
5. Discuss when and how formal methods can be effectively used in the development process.

### FPL-Design: Design Principles of Programming Languages

***Non-core:***

1. Language design principles
    a. Simplicity
    b. Security (See also: SEC-Coding)
    c. Fast translation
    d. Efficient object code
    e. Orthogonality
    f. Readability
    g. Completeness
    h. Implementation strategies
2. Designing a language to fit a specific domain or problem
3. Interoperability between programming languages
4. Language portability
5. Formal description of a programming language
6. Green computing principles (See also: SEP-Sustainability)

***Illustrative Learning Outcomes:***
***Non-core:***
1. Understand what constitutes good language design and apply that knowledge to evaluate a real programming language.

#### FPL-SEP: Society, Ethics, and the Profession

***Non-core:***
1. Impact of English-centric programming languages
2. Enhancing accessibility and inclusivity for people with disabilities – Supporting assistive technologies
3. Human factors related to programming languages and usability
   a. Impact of syntax on accessibility
   b. Supporting cultural differences (e.g., currency, decimals, dates)
   c. Neurodiversity
4. Etymology of terms such as “class,” “master,” and “slave” in programming languages
5. Increasing accessibility by supporting multiple languages within applications (UTF)

***Illustrative learning outcomes:***
***Non-core:***
1. Consciously design programming languages to be inclusive and non-offensive.

## Professional Dispositions

1. **Professional:** Students must demonstrate and apply the highest standards when using programming languages and formal methods to build safe systems that are fit for their purpose.
2. **Meticulous:** Attention to detail is essential when using programming languages and applying formal methods.
3. **Inventive:** Programming and approaches to formal proofs is inherently a creative process, students must demonstrate innovative approaches to problem solving. Students are accountable for their choices regarding the way a problem is solved.
4. **Proactive:** Programmers are responsible for anticipating all forms of user input and system behavior and to design solutions that address each one.
5. **Persistent:** Students must demonstrate perseverance since the correct approach is not always self-evident and a process of refinement may be necessary to reach the solution.

## Mathematics Requirements

**Required:**
- Discrete Mathematics – Boolean algebra, proof techniques, digital logic, sets and set operations, mapping, functions and relations, states and invariants, graphs and relations, trees, counting, recurrence relations, finite state machine, regular grammar. (See also: MSF-Discrete)
- Logic – propositional logic (negations, conjunctions, disjunctions, conditionals, biconditionals), first-order logic, logical reasoning (induction, deduction, abduction). (See also: MSF-Discrete)